

SESSION SUMMARY

Advanced Claude Code with Antigravity

Day Two — Advanced Sprint

Mentor

KVS Dileep — Head of Generative AI Education, Outskill

Host

Kartik — Program Manager, Outskill

Live Build

Sketch-to-App — a Lovable-style clone of reduced scope

ABOUT THIS DOCUMENT

A cleaned, hierarchical reconstruction of the second day of the Claude Code × Antigravity sprint. The live build demonstrated a deeper orchestration pattern — Claude.md for system prompts, memory.md for state persistence, and a stack of curated skills running in sub-agent mode to take a hand-drawn sketch all the way to a working React application.

Framing Day Two

Day two of the Claude Code sprint opens in a noticeably different register from day one. The intermediate foundations have been laid. The mentor no longer needs to explain what Claude Code is,

why Antigravity is in the workflow, or how terminal commands work. Today the energy is pointed forward, into the advanced territory that the first day deliberately held back.

The session is framed with a Marvel Cinematic Universe analogy. A learner who missed day one can still follow along and get value from day two; the narrative will make sense on its own. But the two sessions together form a richer arc, and the mentor encourages anyone who skipped the first to return to it after the second. The recording availability issues are acknowledged openly — the team is working through them, and patience is requested.

What today is, and what it is not

The mentor restates several of the ground rules from day one, because they continue to apply and because today's density makes them even more important. Questions are taken selectively. Tangential doubts are skipped. The session will move faster than a learner might instinctively want, and that is deliberate. The value of a sprint is that it is allowed to be dense; a session that slows to accommodate every question is a session that cannot go deep on anything.

There is one consistent piece of advice the mentor keeps returning to throughout the day: if at any point the session starts to feel overwhelming, do not panic, do not try to keep up at that exact moment. Watch. Absorb what is happening. Return to the recording later, pause at the relevant points, and rebuild the steps in a quieter context.

THE PROMISE OF DAY TWO

The orchestration pattern the mentor actually uses in his own work — not the simpler pattern shown yesterday.

Two new files that change how Claude Code behaves across sessions: `Claude.md` and `memory.md`.

A deeper stack of skills, used in combination rather than alone.

A live build of a sketch-to-app tool with a real drawing canvas.

Practical techniques for token economy, context degradation, and memory persistence.

Quick recap of day one

Before any new material is introduced, the mentor recaps the workflow established the previous day: Claude Code as the agent, Antigravity as the editor and backup, skills as recipes, MCP servers as tools, and the agent itself as the chef that brings the two together. The Model Council application built on day one is referenced, and the mentor acknowledges with visible appreciation that several learners have already deployed their own versions and shared them publicly. Today's build will push that same pattern further.

Claude.md — The Always-Loaded System Prompt

The first concrete advanced technique introduced on day two is the Claude.md file. This is the single most consequential addition a builder can make to their Claude Code workflow, because it reshapes how the agent behaves across every interaction inside a project.

What Claude.md is

Claude.md is a Markdown file that lives at the root of a project and is automatically loaded by Claude Code at the start of every session. Its contents become part of the context the agent sees before it reads the user's first message. In practical terms, it is a preloaded prompt — a sticky piece of text that is always there, always influencing how the agent reasons about the work.

The mentor is careful to distinguish this from retrieval-augmented generation. Claude.md is not a knowledge base the agent searches; it is prompt content that is copied into the conversation every time. That has consequences. Anything in Claude.md costs tokens on every turn. Anything in Claude.md also reliably shapes the agent's behaviour, because it is always visible.

What belongs in Claude.md

Project rules, behavioural guidelines, coding conventions, the stack being used, the philosophy the project follows — these are the kinds of things that belong in Claude.md. Information that needs to persist across sessions, that needs to be true regardless of which specific task the agent is doing, that represents the project's long-term identity.

THE KEY DISTINCTION

Claude.md holds the things that do not change across the life of the project. It is the project's operating agreement with the agent. Dynamic state — what has been done, what is in progress, what is pending — belongs elsewhere, in a companion file called memory.md.

A well-regarded starting template

Rather than writing Claude.md from scratch, the mentor reaches for an established reference. He cites Andrej Karpathy, one of the most respected practitioners in AI, who published a public set of notes from his own Claude Code workflow. Another contributor translated those notes into a sample Claude.md file that encodes behavioural guidelines designed to reduce common LLM coding mistakes.

The mentor reads out the tradeoff the template explicitly declares. The guidelines bias towards caution over speed — they tell the agent to think before coding, favour simplicity, make surgical changes rather than sweeping ones, and stay focused on the stated goal. For trivial tasks this can feel slower than necessary, and the template itself recommends using judgment. For non-trivial work, the caution pays back in fewer regressions and cleaner diffs.

The behavioural rules encoded in the template

- Think before coding — pause to plan rather than charging into implementation.
- Simplicity first — prefer the smallest change that solves the actual problem.
- Surgical changes — edit only what needs editing; do not rewrite adjacent code that is not broken.
- Goal-driven execution — stay focused on the stated objective; do not drift into related-but-unrequested work.

Installing the template

The mentor opens the Sketch2App project folder, launches a Claude Code terminal inside it, and pastes the Karpathy-derived Claude.md URL into the chat with the instruction: set this up as my Claude.md. The agent fetches the content, writes the file, and asks whether the scope should be global or local. The mentor chooses local — the file applies only to this project.

Global versus local — when each is right

The global-versus-local choice is one the mentor keeps returning to across the session, because it trips learners up repeatedly. His position is deliberate. During teaching sessions he always installs locally, because the alternative would pollute every other project he runs with the teaching configuration. For a builder working on their own projects, global installation is often the right choice for the Claude.md file and for skills that are genuinely universal — their presence everywhere saves the effort of reinstalling for each project.

THE MENTOR'S METAPHOR

Think of Claude.md like a tablet prescribed by a doctor. You do not need to understand the exact chemical composition before taking it. Trust the prescription, take the tablet, and pay attention to how the treatment affects the patient. The theory can come later.

Memory.md — The Running Log of the Project

Where `Claude.md` holds the things that do not change, `memory.md` holds the things that do. It is a living document that tracks what has been built, what is currently being built, what is pending, which decisions have been made, and which questions are still open. Together, the two files form the backbone of the mentor's advanced workflow.

The problem `memory.md` solves

The problem is familiar to anyone who has had a long Claude Code session: at some point, either because the weekly token allowance is exhausted or because the context window has grown too large to reason well over, the session ends. When work resumes, the agent has no memory of what happened. The rebuild cost is real — the agent has to re-read the codebase, re-derive the architectural decisions, and rediscover which files matter. That rediscovery is expensive.

`Memory.md` collapses that cost. When the next session starts, the agent reads `memory.md` (and `Claude.md`, because it is always loaded) and immediately has a structured summary of what has happened. The rebuild is seconds of reading instead of minutes of exploration.

The structure `memory.md` follows

The mentor asks the agent to create `memory.md` as a running log with a specific shape. Each of these sections serves a different continuity need:

- **Project overview** — one paragraph describing what the project is and what it is for.
- **Key details** — the stack, the entry points, the external services in use.
- **Constraints and non-goals** — what the project is deliberately not trying to do.
- **Completed steps** — what is already in place.
- **In progress** — what is actively being worked on right now.
- **Pending and next up** — the queue of upcoming work.
- **Open questions and decisions** — anything that has been deferred or needs human input.
- **Notes** — catch-all for anything that does not fit the other categories but is worth remembering.

The update cadence

In the basic version of the workflow, `memory.md` is updated manually. The user issues a command like `update memory.md` at the end of each major step, and the agent rewrites the file to reflect the new state. It is a small discipline that pays back enormously if a session ends unexpectedly.

Later in the session, the mentor introduces a more automated variant — Claude Mem — that captures session state without manual intervention. That evolution is covered in its own section below.

CLAUDE.MD VS MEMORY.MD IN ONE SENTENCE EACH

Claude.md — what this project is, permanently.

memory.md — what has happened in this project, updated as work progresses.

The Advanced Skills Stack

With the two memory files in place, the mentor begins assembling the skill stack that will drive the day's build. Three skills, each with a distinct role, compose the working set. This is a different set from the PM Skills used on day one — that choice was pedagogical; today's choice reflects the mentor's actual daily practice.

Superpowers

The centrepiece of the stack is a skill called Superpowers. Its purpose is to convert good software engineering habits into reusable skills for AI coding agents — so instead of asking the agent to do ad-hoc coding and hoping for the best, Superpowers imposes a structured end-to-end development workflow on the work.

The workflow Superpowers encodes covers brainstorming and requirements clarification, design document creation, test-driven development, and structured execution. The key phrase the mentor emphasises is that the learner does not need to understand each component in detail; it is enough to understand that the skill borrows the judgment of senior software engineers and applies it patiently, step by step, to the build.

The social proof for Superpowers is substantial. Roughly 13,000 forks and 152,000 stars on GitHub — numbers that indicate widespread, sustained use among builders serious enough to vet the skills they install. The mentor has this one installed globally, because it is part of his everyday work.

UI UX Pro Max

The second skill is UI UX Pro Max, which does exactly what its name implies: it brings a disciplined approach to user interface and experience decisions into the agent's behaviour. Around 65,000 stars and 6,500 installs give it similar credibility to Superpowers.

The mentor installs it locally for this project. He issues a short instruction — check if the skill is installed, and if not, install it — and the agent handles the marketplace lookup, the plugin installation, and the reload. The workflow illustrates a pattern worth internalising: instead of manually running installation commands, describe the desired end state and let the agent figure out the steps.

SuperDesign, reused

The third element is SuperDesign, which was introduced on day one and returns today in a complementary role. It provides an overall design-language scaffolding that sits alongside the more granular component choices made through a different source introduced later.

ORCHESTRATION, NOT COLLECTION

The point is not to install many skills. The point is to install a small number of complementary skills whose capabilities do not overlap. Two skills that do the same thing will confuse the agent about which to invoke. The mentor deliberately rejects redundancy — one skill per role, selected for quality, combined with judgment.

The trade-off being made

The mentor is explicit that this stack is not optimised for tokens. It is optimised for the quality of the resulting application. A bigger skill set means more context loaded at start-up and more tokens consumed per turn, and that is acceptable because the return — a well-architected build with tests, security considerations, and proper design discipline — is worth the spend. Token optimisation techniques are covered separately; they are not a reason to weaken the skill stack.

Component-Level Design with 21st.dev

On day one, SuperDesign was used to pick a single design language for the entire Model Council application — one aesthetic, applied everywhere. Day two introduces a more granular alternative for builders who want to control the look of each section independently.

The resource

21st.dev is a catalogue of design components organised by function: announcements, backgrounds, borders, calls to action, documentation, features, footers, hooks, images, maps, navigation menus, pricing sections, and many others. Each component ships with a copy-ready prompt that can be pasted into Claude Code to produce that specific visual element.

The mentor's summary of the choice between the two resources is practical. SuperDesign is right when the builder has a clear overall aesthetic in mind and wants one cohesive system. 21st.dev is right when the builder has opinions about each section — when the navbar should animate in a particular way, or the background should use a specific shader, or the pricing card should have a very particular layout.

Using them together

During the Sketch-to-App build, the mentor uses both. He asks the agent to identify the proper sections of the application and produce search terms for 21st.dev — navbar, sketch workspace, preview iframe, model picker — so that component-level choices can be made where they matter, while SuperDesign handles the overall feel.

Rather than manually searching 21st.dev for each term and pasting prompts one at a time — a tedious and token-heavy process — he delegates the selection to the agent itself, asking it to pick the right components using the skill's best practices. The skill knows what quality components look like. Trust the skill.

WHEN TO PICK EACH TOOL

SuperDesign — when the whole app should feel like one thing. Fast. Coherent by default.

21st.dev — when each section should feel like itself. Slower. Gives precision where precision matters.

Both, together — when the overall vibe is clear but a few sections deserve special care.

The Live Build: Sketch-to-App

The project built on day two is a sketch-to-app tool — something like a Lovable clone at smaller scope. The user either draws a wireframe on an in-browser canvas or uploads an image of a hand-drawn sketch, and the application generates working code from it. The mentor has not built this before; the build happens live in front of the audience.

Brainstorming the application

The process begins with the Superpowers skill taking over. Instead of a PRD in the day-one sense, the skill runs a brainstorming dialogue with the user, asking questions that any competent product manager would ask before any code is written. Each answer sharpens the specification.

Input format

The first question offers three interpretations of sketch-to-app: upload a hand-drawn wireframe, draw directly in an in-browser canvas, or upload a Figma-style mockup. The mentor picks both image upload and in-browser drawing, with a canvas as the second input mode.

Output format

Options range from a single HTML file to a React-plus-Tailwind download to a live sandboxed preview inside the app itself. The mentor chooses all of the above, letting the user pick the format at generation time.

Model choice

The application needs a vision-capable model for the sketch-to-code translation. With OpenRouter as the API provider, any of Claude Opus, GPT-5, or the latest Gemini are accessible. The mentor specifies that the user of the built application should be able to pick the model — another small decision that enlarges the product's range.

Deployment target

Four options are offered: local-only dev tool, deployable web app with user-supplied API keys, deployable web app with hosted keys and per-user limits, or local-first-then-deployable. The mentor chooses local-first. This is a deliberate scope decision — the first version is a developer tool running on the mentor's own machine, with deployment deferred as a future phase.

Canvas library

The brainstorm surfaces the question of which canvas implementation to use. TLDraw and Excalidraw are both surfaced as options. The mentor chooses TLDraw, which offers a clean, developer-friendly canvas experience suitable for sketching user interfaces.

Iteration model

How should the user refine a generated app? Two options: chat-based refinement (describe what to change and the model regenerates) or re-sketching on top of the existing preview (annotate the preview with TLDraw and regenerate). The mentor chooses both, starting with chat refinement and leaving re-sketching as a future addition.

V1 scope

Finally, the skill forces a scope decision. Three options are presented: a minimal V1 (upload plus TLDraw, generate HTML, preview, download, chat-refine, skip React sandbox and re-sketching), a fuller V1 (upload plus TLDraw, all three input formats including Figma-style mockups), or a full-vision V1 with every feature enabled. The mentor picks the middle option — upload plus TLDraw, sensible defaults — with future iterations available to add the rest.

THE DISCIPLINE OF THE BRAINSTORM

Every question the skill asks is one the builder would otherwise have to answer later, often in the middle of implementation when the cost of changing direction is highest. The brainstorm is not paperwork; it is a way of front-loading every decision that the code will eventually depend on.

Architecture and Design Documents

With the brainstorm complete, Superpowers transitions into producing a design document. This is a denser artefact than a PRD — it captures the architectural decisions, the data flow, the tech stack, the signature components, and the trade-offs that were considered and rejected.

Architecture options

Three candidate architectures are surfaced:

Option one — Next.js all-in-one

A single Next.js application with the server routes proxying to OpenRouter and the client hosting the TLDraw canvas, the upload control, and the chat panel. State lives in React. Deployment to Vercel is a single click. This is the recommended option — simplest, fewest moving parts, and it protects the OpenRouter API key on the server from day one.

Option two — split front and back ends

A lightweight front-end paired with a separate back-end service. More flexibility to swap the front end independently, but more configuration and a larger surface area to secure.

Option three — pure client side with local storage keys

Everything runs in the browser with the user's OpenRouter key held in local storage. Simplest to deploy but unsafe — any front-end code has access to the key, and the security implications rule it out for anything beyond the most casual demo.

The chosen architecture

The mentor picks option one. Next.js, server-side proxying, React state, one deployment target. The skill then produces a full architecture document — input handling, output handling, the data flow between canvas and model, the tech stack, the visual language, typography, desktop layout, signature components, API contract, error handling, accessibility considerations, and an explicit out-of-scope section for future phases.

Writing the design into the project

The design document is written into a dedicated plans folder at the project root. This is a Superpowers convention — plans live in their own folder, memory lives in its own file, and the code itself lives

separately from both. The separation keeps each artefact readable and makes it easy for later sessions to pick up exactly where they left off.

Before implementation begins, the mentor asks the agent to update `memory.md` with the current state — the PRD-equivalent, the design document, the open decisions. This is the continuity checkpoint. If the session had to end here for any reason, the next session could resume from the same state simply by reading `memory.md` and the plans folder.

The Admin Mode: dangerously-skip-permissions

With the design locked and `memory.md` current, the build enters implementation. Before starting, the mentor introduces one of the most consequential flags in the Claude Code toolkit — and one that must be used carefully.

What the flag does

When Claude Code is launched with the `--dangerously-skip-permissions` flag, the agent stops asking for permission before every file write, every command execution, every package install. It operates in what the mentor calls admin mode — it decides, it acts, and the human does not interrupt.

THE TRADE-OFF

Admin mode trades safety for speed. The safety checks were there for a reason: they let the human catch unintended actions before they happen. Skipping them means accepting that the agent will do what it thinks is right without a second opinion. For well-scoped work inside a sandboxed project folder this is usually fine. For a first-ever Claude Code session, or for work inside a folder that contains important files, it is not.

When the mentor uses it

In the live demonstration, the flag is used deliberately. The plan has been built. The design is approved. The steps are well-defined sub-agent tasks in a clean project folder. There is no ambiguity about what needs to be done, and interrupting thirty times to approve each file write would destroy the pedagogical flow. Admin mode lets the build run end-to-end while the mentor narrates what is happening.

He is explicit that this is a pattern for known-good workflows, not a default mode. For a newer builder who does not yet have a calibrated sense of what the agent is likely to do, leaving the permission

prompts on is the safer habit — at least until the workflow has been exercised enough times to be trusted.

Exiting and re-entering

The mentor exits the current Claude Code session before launching the admin-mode version. The exit is deliberate — it ensures the agent is no longer holding any stale context from the brainstorm and design phases. Only `memory.md`, `Claude.md`, and the `plans` folder persist to the new session, and those are exactly the right things to persist.

Sub-Agent-Driven Execution

Superpowers offers two execution modes for the implementation phase: a single-threaded sequential run, or a sub-agent-driven run where multiple agents spin up in parallel to handle independent tasks. The mentor chooses sub-agent-driven for the live build, because it showcases what advanced orchestration actually looks like.

What sub-agents do

A sub-agent is a spawned worker agent that takes one bite-sized task, executes it, reports back, and terminates. The top-level agent coordinates — it reads the plan, hands tasks to sub-agents, waits for completion, checks the result, and moves on. For parallelisable work this is significantly faster than doing everything in a single thread, because the work genuinely happens at the same time rather than one step after another.

The plan structure

Superpowers breaks the build into thirty bite-sized tasks spread across nine phases. Task 0.1 is git initialisation. Task 0.2 is dependency configuration. Subsequent phases cover scaffolding, the prompt builder, the canvas integration, the model adapter, the preview pipeline, and so on. Each task is small enough that a single sub-agent can complete it in a focused run.

The execution cadence

Two cadences are offered: dispatch each task sequentially with an auto-continue flag (the next task starts as soon as the previous one finishes), or dispatch with pauses for human review at each step. The mentor picks auto-continue with pauses only if a review fails — the build runs itself until something needs human judgment.

WHY THIS WORKS

Each sub-agent operates on a narrower context than the top-level agent would. The narrower context is more reliable and cheaper to run. The coordination overhead is outweighed by the accuracy gains from keeping each piece of work in a focused scope. This is the same pattern that scales in human organisations — divide the work, staff each division with a specialist, coordinate at the top.

Updating memory at checkpoints

Throughout the sub-agent execution, `memory.md` continues to accumulate state. Each completed phase is logged; each decision is recorded; each unresolved question is parked for the human's later attention. The file becomes a running diary of the build, readable at any moment by any agent that needs to catch up.

Context Rot: The 50 Percent Rule

One of the most concrete pieces of advice in the entire session is the 50 percent rule. It deserves its own section because it shapes nearly every long-session workflow decision a builder makes.

The rule

If the current session's context usage climbs above fifty percent, Claude Code's performance begins to degrade noticeably. The agent makes more mistakes, forgets earlier decisions, produces lower-quality code, and needs more prompting to stay on track. This is not a soft preference — it is an observable pattern that the mentor has experienced repeatedly and that aligns with broader research findings.

The technical phenomenon has a name. The mentor references a research paper titled Context Rot, and points learners who want to go deeper towards it. The short version: large language models are not equally good at reasoning over all parts of their context window. As the window fills, the middle of the context becomes less reliable, and the model's overall behaviour degrades. Fifty percent is a practical threshold the mentor has observed, not a hard cutoff.

What to do when the threshold is hit

Step	Action
Pause	Finish the task currently in progress. Do not interrupt mid-step.
Persist	Ask the agent to update memory.md with the current state — what is done, what is next, which decisions were made.
Exit	Close the Claude Code session.
Re-enter	Launch a fresh session. Claude.md and memory.md are automatically available to the new session.
Resume	Ask the new agent to read memory.md and the plans folder, and continue from where the previous session left off.
THE BIGGER IDEA A long session is not inherently more productive than a sequence of short sessions, as long as the continuity files are kept current. Restarting a Claude Code session is cheap when Claude.md and memory.md preserve the project's state. The expensive thing is pretending the context window is infinite when it is not.	

How to check the usage

The mentor defers the mechanics of tracking context usage until later in the session. For now, he flags that Claude Code's terminal interface displays current-session context usage at the bottom of the window as a percentage, and that a customisable status line can make the reading more prominent. Specific instructions follow in the tips-and-tricks section at the end of the session.

Token Economy and the Clarity Principle

Tokens are the currency of this work, and the single biggest determinant of how far they stretch is the clarity the human brings to the task. The mentor makes this argument multiple times across the session, because it is consistently underweighted by newer builders.

Why clarity beats any optimisation

When the human is vague, the agent does exploratory work — it asks clarifying questions, it writes code that turns out to be wrong, it produces approaches that have to be thrown away. Each of those rounds costs tokens. A sharp specification up front avoids all of them. Every token spent on the brainstorm and design document was repaid multiple times over when the implementation phase proceeded without detours.

The Sketch-to-App build consumed many tokens during the brainstorm — every answer to every question produced more design documents — but those tokens were invested, not spent. They bought a plan that the implementation phase could follow mechanically.

THE CARDINAL RULE, STATED AGAIN

No packages, no skills, no magic wand can replace clarity about what you are trying to build. The single highest-leverage thing a builder can do before writing any prompt is to decide, concretely, what the finished thing looks like.

Caveman skill — a cautionary tale

Someone had reached out to the mentor on LinkedIn about a skill called Caveman, which reduces token usage by stripping natural language down to its barest nouns and verbs. The example claim is that a normal Claude prompt of 69 tokens can be reduced to 19 tokens in Caveman style — a seventy-two percent reduction.

The mentor is openly sceptical. Natural language carries nuance, and that nuance is what makes the agent's output precise. Stripping articles, prepositions, and connective tissue saves input tokens at the cost of reducing the context the agent has to reason with. The output quality suffers, sometimes dramatically, and the saved tokens are reclaimed by the extra rounds needed to recover from misinterpretation.

The right way to think about it

Do not hyper-optimize tokens at the cost of clarity. Spend the tokens that need to be spent to make your intent unambiguous. Save tokens by restarting sessions with good memory files, by using lighter models for mechanical phases, and by letting Claude Mem (discussed next) handle context persistence automatically. Those are token savings that do not cost you anything in build quality.

Claude Mem: Automated Memory Persistence

Memory.md works, but it depends on the human remembering to issue update commands at the right moments. The mentor introduces a skill that makes this automatic and, in his own words, provides the single biggest unlock available for managing long Claude Code projects.

What Claude Mem does

Claude Mem hooks into Claude Code and automatically captures what is happening as the agent works. When a session ends — whether deliberately or because tokens ran out — Claude Mem has already persisted the state. When the next session starts in the same project, the memory surfaces without any explicit prompt; the agent resumes with full context.

The mentor demonstrates this by reopening the Model Console project from day one. When Claude Code launches inside that folder, Claude Mem reports a 91 percent reduction in context reconstruction, because the skill already knows what has been built: PM Skills installed, workplace catalogue exported, Model Council PRD created, Bold Editorial Studio design system added, design system specification, pre-built environment, implementation approach, OpenRouter API configuration, global styles. Every significant step of the previous session is there, automatically.

Why this matters more than caveman-style optimisation

The Caveman skill saves tokens on the input side by stripping language. Claude Mem saves tokens by never having to reconstruct state in the first place. The savings are larger, the clarity is preserved, and the human does not have to remember to update anything. This is the kind of automation the mentor favours — something that removes a chore without compromising quality.

THE WORKFLOW WITH CLAUDE MEM INSTALLED

Install the Claude Mem skill once, globally.

Work in Claude Code normally. Claude Mem captures state in the background.

When you return to a project — the next day, the next week, after an outage — launch Claude Code in that folder and the memory surfaces automatically.

No manual update-memory.md command is needed. The skill has already done the work.

Claude Mem and memory.md together

The two are not in competition. Claude Mem is a boost to memory.md, not a replacement. Memory.md remains useful as the human-authored layer — the place where intentional decisions, open questions, and architectural notes live in a form the human has curated. Claude Mem adds the automatic layer underneath — the fine-grained record of what happened, generated without intervention. The human controls the strategic narrative; the skill captures the operational detail.

Testing with End-to-End Skills

With the core application built and the OpenRouter key in place, the mentor turns to testing. Shipping something without tests is a pattern the sprint explicitly warns against, and there are high-quality skills available for test generation that make the step relatively cheap.

Finding the right skill

The mentor opens skills.sh and searches for testing. Multiple options appear. Web app testing by Anthropic itself is one. An E2E Testing Patterns skill is another, with around 33,500 GitHub stars, indicating sustained adoption. The mentor picks the E2E Testing Patterns skill because end-to-end testing — simulating real user interactions through the full application stack — is the most useful kind of coverage for a tool like Sketch-to-App.

How to judge a testing skill

Signal	What it tells you
GitHub stars	Breadth of interest. Higher numbers suggest the skill has been vetted by many builders.
Install count	Depth of adoption. A skill with many installs is being used, not just admired.
Source	Is the author a recognised practitioner or organisation? Anthropic, Google, known community figures raise trust.
Recency of updates	A skill that has not been updated in a year may be tracking a version of Claude Code that has moved on.
Description clarity	A skill whose description is vague or confusing often reflects vague or confusing implementation.

What the skill installs

The E2E Testing Patterns skill brings Playwright into the project — a modern browser-automation framework suitable for simulating real user flows. It also writes a first set of critical-path tests based on the project's architecture, runs them, and produces a report. The pattern is consistent with Superpowers: the skill knows what good testing looks like, the human does not have to.

The testing session cut short

During the live build, the end-to-end testing run consumes enough GPU on the mentor's MacBook M1 Pro to freeze Zoom mid-session. He kills the testing process to restore control of the call and acknowledges, with some humour, that this is a reminder that local testing is a real resource consumer. For learners running on more modest hardware, end-to-end test runs may need to happen outside the live session context.

Security Audits

Testing verifies that the application does what it is supposed to do. Security audits verify that it does not do what it is not supposed to do. The mentor places strong emphasis on this step, particularly for builders coming from non-technical backgrounds.

Why security matters more for vibe coders

A developer with formal training has internalised a sense of where vulnerabilities tend to live — injection attacks, authentication weaknesses, exposed secrets, unvalidated inputs. A vibe coder — someone building through natural-language instruction to an agent — may not have that sense yet. Which means the agent needs to be the one applying the audit, using a skill that knows what to look for.

Finding a security skill

The mentor searches skills.sh for security. A Security Best Practices skill with around 32,800 stars surfaces as the leading candidate — high adoption, mature, well-regarded. He contrasts it with newer alternatives that have much smaller star counts and less-convincing descriptions, which he passes over. The lesson: when it comes to security, lean towards widely-vetted tools rather than novelty.

THE RULE FOR SECURITY-CRITICAL SKILLS

Stars and install counts matter more for security skills than for almost any other category. A buggy UI is annoying. A buggy security audit is dangerous. Pick the skill that thousands of other builders have exercised in anger.

Semgrep as an alternative path

Towards the end of the session, a question comes in about alternatives if a Claude Code skill cannot handle a particular security concern. The mentor recommends Semgrep — a static analysis tool with a subscription tier and its own MCP server. He notes that Lovable's vibe-coding security pipeline is actually powered by Semgrep, which is a useful credibility signal. For serious security work that exceeds what a general-purpose skill can handle, Semgrep is the right next step.

API key protection during the build

Before running the dev server, the mentor stops screen sharing to paste his OpenRouter API key into the local `.env` file. The ritual is the same as day one: never let a key appear in chat or on a shared screen. Treat every key as one screenshot away from compromise if it is visible, and generate a new key if one is ever exposed.

Running the App: The Sketch-to-App Demo

With dependencies installed, design applied, implementation complete, tests scaffolded, and the API key in place, the mentor starts the dev server. The Sketch-to-App web application loads in a browser window.

The interface

The interface reflects the design decisions made during the brainstorm. A top bar with model selection. A left panel containing the TLDraw canvas workspace. A generate button. A right panel where the generated preview will render. An image-upload alternative to the canvas for users who prefer to sketch on paper and photograph the result.

The first drawing

The mentor begins by sketching a simple login screen directly on the TLDraw canvas — a title reading Welcome to Outskill, two input fields labelled User and Password, and a generate button. He clicks Generate. The model receives the canvas image, interprets it, and produces code.

The rendering is not perfect. There is some overlap, some misinterpretation of element placement. But the core structure survives — the title is recognised, the two input fields are generated, the submit button is produced. For a first-time build of a sketch-to-code tool on an afternoon with a hand-drawn sketch, the result is meaningfully useful.

The second test with an uploaded image

The mentor then switches to image upload mode and feeds the application an image of a more detailed app sketch pulled from a search. He tries Gemini 3 Pro first, runs into a configuration issue he acknowledges will need debugging, and switches to Claude Opus 4.5. The Opus run produces a reasonable approximation of the uploaded sketch.

WHAT THE DEMO PROVES

A concept that looks like commercial tooling — Lovable, V0, similar products — can be approximated in an afternoon by a builder with the right orchestration.

Single-screen sketches translate to code reliably. Multi-screen designs need more iteration.

The scaffolding generated by Superpowers supports incremental improvement. Adding a database, polishing the preview, tightening the code — all of those are small future iterations, not rewrites.

Honesty about the result

The mentor is explicit that what was built is a Minimum Viable Product. It is not production-ready. It will not handle complex multi-screen designs with the fidelity of a dedicated commercial tool. The point was to show that the workflow produces a working prototype of a genuinely complex application class, in one session, using orchestrated skills. The prototype is the starting point for further work, not the destination.

Troubleshooting and Session Hygiene

Several practical issues arose during the live build, and the mentor's handling of each demonstrates the kind of troubleshooting instincts that turn a mid-session disruption into a footnote rather than an ending.

The frozen Zoom call

The most dramatic interruption was the end-to-end testing run consuming enough local GPU to lock up Zoom entirely. The mentor's response was direct: kill the testing process, restore control, explain what happened, and move on. The lesson is that resource-intensive agent operations — test suites, builds, parallel sub-agent runs — are not free on local hardware. A builder with a less powerful machine should plan for this and schedule heavy runs outside of sessions where attention is needed elsewhere.

The failed Gemini generation

When the sketch-to-code run with Gemini 3 Pro returned an error, the mentor did not get stuck debugging it. He noted the problem, moved to Claude Opus 4.5, and completed the demonstration. This is a pragmatic pattern: when a tool choice fails and a working alternative is available, switch first and investigate later.

Stale context in mid-session

Several times during the build, the mentor exits Claude Code and re-enters. Each exit is deliberate. Before a major phase transition (brainstorm to design, design to implementation), a clean context avoids carrying stale assumptions forward. The memory files preserve what matters; the exit discards what does not. This is session hygiene applied consistently.

A PATTERN TO STEAL

Think of Claude Code sessions the way a careful developer thinks about Git commits — small, focused, well-scoped, and closed cleanly once their purpose is done. Do not try to do everything in one long session. Multiple shorter sessions, each with a specific goal, produce better results and cost fewer tokens.

Existing-project workflows

A question from the audience concerned applying this workflow to an existing codebase rather than a greenfield project. The mentor's answer introduces a two-agent pattern: one agent reads the full codebase and produces a structural summary, and a second agent uses that summary plus a plan to make targeted changes. Midway projects are genuinely harder because the context load is heavier, but the orchestration pattern scales — it just requires an initial scanning phase that greenfield builds do not.

He notes that running parallel agents over a codebase is one of the more powerful patterns available, though a full treatment is beyond the scope of this session.

Tracking Tokens in Real Time

The 50 percent rule only works if the percentage is visible. The session closes with a small but practical tip for surfacing context usage at all times.

The Claude Board tips repository

The mentor references a public GitHub repository of Claude Code tips and tricks — around 45 documented techniques for improving the day-to-day workflow. The one he highlights is the very first: customising the status line.

Customising the status line

Claude Code's status line — the line of information at the bottom of the terminal — is configurable. With a small amount of setup, it can display current context usage as a percentage, the model in use, and other live metrics. For a builder who is actively watching for the 50 percent threshold, having that number visible at all times is valuable.

The mentor acknowledges that the setup is slightly advanced and points learners towards the repository itself for detailed instructions. He offers a general warning: the tips repository is dense, and a learner who feels overwhelmed should not try to apply all 45 at once. Pick one, adopt it, wait for it to become habitual, and then consider the next.

A GOOD FIRST TIP TO ADOPT

The customised status line is useful on day one and pays back on every session thereafter. If only one tip from the repository is going to be adopted, this is the right one. Token visibility is the feedback loop that makes every other discipline work.

The Philosophy of Skill Curation

Across the two days of the sprint, the mentor has returned again and again to the same meta-question: how do you choose which skills to install? The question deserves its own consolidated treatment, because the quality of a builder's skill library determines the quality of everything they build.

The selection criteria, consolidated

- **Stars and forks** — the coarse measure of adoption. A skill with few stars may be excellent and simply new, but unless the author is already known, the risk is higher.
- **Author reputation** — a skill published by Anthropic, Google, or a recognised community practitioner comes with earned credibility.
- **Install count** — distinct from stars. Stars measure breadth of interest; installs measure depth of use.
- **Fit to the specific task** — the best skill in a category is not always the right skill for your current build. Ask Claude Code to read the skill description and tell you whether it fits.
- **No conflicts** — never install two skills that cover the same ground. The agent will not know which to invoke, and the output quality will suffer unpredictably.

Global versus local, refined

The mentor's position on scope is more nuanced than it might first appear. Skills the builder uses on every project — Superpowers, Claude Mem, perhaps a personal coding-standards skill — belong globally, because the cost of reinstalling them for each project is not worth the marginal benefit of isolation. Skills that serve a specific project or domain — UI UX Pro Max for UI-heavy work, testing skills for projects with testing needs — belong locally, where they cannot clutter other projects.

THE RULE TO AVOID GETTING CONFUSED

Global for always-useful skills. Local for sometimes-useful skills. Never two skills that do the same thing in the same scope. When in doubt, install locally — promoting to global later is easier than the reverse.

Evaluating a skill before installing

The mentor describes a pattern he uses that the audience should adopt: paste a candidate skill's URL into Claude Code along with a description of the project, and ask whether this skill is relevant. The agent reads the skill's documentation and gives an honest take. Most skills are either broadly applicable or narrowly targeted, and the agent is generally good at distinguishing which is which. If the skill is not a fit, skip it — there is no virtue in a crowded library.

Questions and Answers

The session closes with a live Q&A. The selection below captures the substantive exchanges that add value beyond what was already covered in the main session — ordered by topic rather than chronology.

On workflow choices

- **Claude Code plugin versus terminal** — not much practical difference for most work. The terminal is more lightweight and supports running several parallel terminals at once. Multiple plugins inside a single editor can destabilise the environment.
- **PM Skills versus Superpowers** — start with PM Skills to get hands-on familiarity, then move to Superpowers once the basic rhythm is established. PM Skills is simpler to learn from; Superpowers is more powerful to build with.
- **Spec-driven development as a separate session** — not planned. The work shown today is already a form of spec-driven development — a software-engineering-driven process where the specification is generated before the code. The pattern can be adopted without a dedicated deep-dive.

On publishing and scaling

- **How to publish the finished app** — push to GitHub; from GitHub, a single click deploys to Vercel or Netlify. No manual server configuration required for most prototypes.
- **Is this workflow scalable** — for prototypes, yes. For production systems serving millions of users, this workflow produces the starting point, not the finished infrastructure. Scaling to that level requires additional systems work beyond Claude Code.
- **Claude Code versus Lovable** — both target the same outcome (working applications from natural-language input), but Claude Code with orchestrated skills gives the builder much more control over the process. Lovable is faster to a first result; Claude Code produces a more customisable result.

On the day-to-day practice

- **Where does the mentor find his resources** — no single consolidated source. Two to three hours of reading per day, mostly Twitter and newsletters. Roughly one in ten things he reads is eventually shared with students; the other nine are either still being evaluated or turned out to be less useful than they first appeared.
- **Could Claude rebuild skills by reviewing skill websites** — technically yes, but the mentor prefers the search-and-evaluate workflow because it keeps him in touch with what the community is producing.

- **Security of installing skills from skills.sh** — not automatic. Each skill should be reviewed before installation, especially ones with low star counts or from unrecognised authors. The same due diligence that applies to installing any open-source package applies here.

On memory and context

- **Claude Mem versus manual memory.md** — Claude Mem is automatic and continuous; memory.md is under explicit human control. The two are complementary. Claude Mem captures operational detail without intervention; memory.md captures strategic decisions the human wants foregrounded.
- **Skills in an existing project** — one skill can read and understand the code; other skills can take that understanding and make changes from there. The workflow generalises to brownfield projects, though the initial scanning phase adds cost.

On tooling comparisons

- **Antigravity agent's Gemini tokens** — exhaust quickly under heavy use. Every agent tool has the same fundamental constraint; no provider offers unlimited reasoning. OpenAI Codex currently offers slightly more headroom on similar workloads because OpenAI is in catch-up mode with Anthropic.
- **OpenClaw versus Claude Code** — different tools for different purposes. The mentor generally declines to compare tools that serve different roles — each has its place.
- **Python coding skills** — not a category the mentor recommends directly. For coding work that needs best-practice documentation lookup, Context7 is the skill to install — it provides up-to-date documentation access for the libraries the agent is working with.

Closing: Learning for Life, Not Coursework

The session closes on a note that reframes the entire sprint series. A learner asks what the follow-on coursework looks like, expecting a structured path of further programs. The mentor's answer reframes the question.

There is no more coursework

This is no longer coursework. It is learning for life. The sprints and masterclasses will continue, appearing every two to three weeks with different topics, different mentors, and different depths. But the relationship between the learner and the material has shifted. The learner now has enough foundational

fluency to absorb new material as it arrives, explore it, and decide whether to adopt it. The teacher is no longer prescribing; the community is sharing.

What comes next

Sprints will typically be multi-session, like this one, exploring a single topic across two or more consecutive days. Masterclasses will be standalone sessions on narrower topics. Newer mentors will appear over time, bringing fresh perspectives. The feedback learners provide after each session determines which mentors return.

In-person meetups are being discussed but not yet scheduled. The mentor hopes to meet learners in person at some point, and invites anyone with ideas for community initiatives to reach him on LinkedIn.

THE PARTING MESSAGE

The complete framework has been given. Claude.md for always-loaded context. Memory.md and Claude Mem for continuity. Superpowers for structured building. UI UX Pro Max and 21st.dev and SuperDesign for design. E2E Testing Patterns for tests. Security Best Practices for audits. The chef-recipe-tools model for thinking about it all.

Now explore. Build things. Break things. Come back to the next sprint with questions that came from actually trying the workflow.

The mentor has done his part. The next move is yours.

The Catalyst programme

Towards the end of the session, the mentor briefly introduces the Catalyst programme — an ongoing community-based follow-on for learners who want to go deeper than the sprints alone. The pitch is short and unpolished, deliberately so. The programme focuses on deepening a chosen skill, building systems on top of it, and taking those systems to market — go-to-market strategy, packaging, pricing, landing pages, all of the machinery around a technical capability.

The mentor is explicit that Catalyst is not for everyone. It involves less hand-holding than the programs most of the audience has come from. The framing is a gym: the environment is provided, the coaching is available, but the work itself is the learner's to do. The curriculum covers problem-solving with AI, AI employees (landing pages, prototypes, sales agents, RAG agents), weekly business report generators, advanced RAG, context engineering, voice agents, automation and agent integration, packaging and selling AI agents, basic UI/UX design principles, pricing and revenue planning, lead magnets, delivery SOPs, and deeper Claude Code sessions beyond what the sprint has covered.

A small number of seats are available in the next cohort. The mentor deliberately leaves the pitch short — there is no deck, no formal presentation, just an open invitation to book a call and see if it is a fit. No obligation. The sprints and masterclasses continue regardless of whether a learner joins Catalyst or not.

Appendix: Quick Reference

The two-day workflow, consolidated

Stage	Action
Frame the project	Write one sentence describing what the project is for.
Create the folder	Fresh folder per project. Open it in Antigravity.
Install Claude Code	Anthropic's official extension, logged in, model picker set to Opus for planning.
Set up Claude.md	Paste a trusted template (e.g. Karpathy-derived). Keep local unless universally useful.
Set up memory.md	Ask the agent to create the file with the standard sections.
Install the skills stack	Superpowers globally. UI UX Pro Max and any project-specific skills locally. Claude Mem globally.
Brainstorm	Let the Superpowers brainstorm skill interview you. Answer every question.
Design doc	Review the generated design document. Approve, or request changes.
Update memory.md	Before implementation, commit the current state to memory.
Admin mode	Relaunch with <code>--dangerously-skip-permissions</code> if the plan is well-scoped.
Sub-agent execution	Auto-continue with review pauses only on failure.

Stage	Action
Watch context usage	At 50 percent, pause, persist, exit, re-enter, resume.
API key	Stop screen sharing. Paste key into .env. Confirm .gitignore excludes .env.
Run the dev server	Test the core flow end-to-end.
Tests	Install an E2E testing skill. Run Playwright against the app.
Security audit	Install Security Best Practices. Audit the code. Consider Semgrep for deeper work.
Ship	Push to GitHub with a clean README. Deploy via Vercel or Netlify.

Skills referenced in day two

Skill	Role in the workflow
Superpowers	End-to-end software-engineering workflow. Brainstorming, design docs, sub-agent execution.
UI UX Pro Max	Structured UI/UX decisions. Installed locally for projects with substantial interface work.
SuperDesign	Overall design-language scaffolding. Single aesthetic applied across the app.
21st.dev	Component-level design library. Granular control of individual UI sections.
Claude Mem	Automatic context persistence across sessions. The single biggest token-saving unlock.
E2E Testing Patterns	Playwright-based end-to-end testing. Writes critical-path tests and runs them.
Security Best Practices	Vulnerability scanning and security audit across the codebase.
Context7	Live documentation lookup for the libraries in use. Keeps coding guidance current.

Skill	Role in the workflow
Claude Board tips	Tips and tricks for Claude Code, including status-line customisation.

Mental models to carry forward

- **Chef, recipes, tools** — Claude Code is the chef; skills are the recipes; MCP servers are the tools. All three need to be present and well-matched for the dish to come out right.
- **Claude.md vs memory.md** — permanent project identity versus running project state. Both files, always.
- **The 50 percent rule** — context over 50 percent means performance degrades. Update memory, exit, re-enter, resume.
- **Clarity over optimisation** — the most expensive thing in any build is ambiguity. Invest tokens in the spec; save tokens during execution.
- **Global vs local** — always-useful skills go global. Sometimes-useful skills go local. Two overlapping skills in the same scope is never correct.
- **Trust the recipe** — skills exist to close the knowledge gap between the builder and the domain expert who wrote the playbook. Use the recipe; do not reinvent it.

The mentor's reading habit, briefly

Two to three hours a day on Twitter and newsletters. Treat it as a research practice, not casual consumption. Expect to find one useful item in ten; let the other nine go without regret. Over months, the compounded effect is a library of ideas that most practitioners do not have, built from reading that most practitioners do not do.

End of summary. End of sprint. Start of the build.